

Sprint 4 Report (Aug 27, 2026 – Sep 30, 2026)

Team 11: Kellam Adams, Melvin Sanare, Kaleb Kebede

Project: Squatch Sports Basketball Training App – Android/Wear OS Expansion and Gamification

Client: Squatch Sports (Mike Niehl, Preston David)

YouTube link of Sprint 4 Video (Make this video unlisted)

<https://youtu.be/DzcnNB6Mgj0>

What's New (User Facing)

Android phone app: The Squatch Sports app now runs on Android, with the Dashboard, Drills, Drill Details, Workout, History, Analytics, Goals, and Settings screens ported from iOS.

Wear OS watch app: Players can log shots on an Android smartwatch by swiping the center pad (up = make, down = miss, right = swish) during a workout started on the phone, with a court view showing where to shoot and a summary at the end.

Streak tracking (Android phone and watch): Current and best training streak, a 7-day calendar strip, progress toward the next milestone, and a celebration popup when a streak starts, continues, sets a new best, or reaches a milestone. One rest day is allowed; two missed days in a row reset the streak.

Goal progress (Android phone and watch): Daily shots, daily makes, weekly sessions, and daily FG% goals shown as animated progress rings. Goals are editable, and completing one shows a trophy popup.

XP and levels (iOS): Players earn XP for completing workouts, makes, swishes, and high shooting percentage, level up every 100 XP, and see their level on a Player Progress card with workout-complete and level-up alerts.

Watch gamification: The watch home screen shows the player's streak and goal rings, the workout summary shows the updated streak, and celebrations appear on the watch with a vibration.

Work Summary (Developer Facing)

Sprint 4 started a new direction for the project. At our September 2 and September 17 client meetings, Squatch Sports explained that our iOS and watchOS work had been released in their production app, so this semester focuses on expanding to Android and Wear OS and adding gamification that makes logging workouts more enjoyable ("hide the broccoli"). Kellam used AI tools, as the client requested, to port the iOS and watchOS app to a Kotlin/Jetpack Compose project with phone (`app`), watch (`wear`), and `shared` modules, keeping the same shot codes and drill data as the Apple version. Melvin set up Android development on Windows for the first time, verified the port in paired phone and Wear OS emulators, and built streak tracking and goal progress on the Android phone and watch, with the phone syncing a streak/goal snapshot to the watch through the Wear OS Data Layer. Kaleb built an XP and leveling system on iOS. Our biggest barriers were the team's limited Android experience, testing streak behavior across multiple days (solved by changing the emulator's date and adding a "Load Demo History" option), and an endless flame animation that made the emulators unresponsive, which we found by measuring CPU usage and fixed by limiting the animation to three pulses. A key lesson was that AI-generated code can build and pass tests while still causing real problems, so every feature needs hands-on testing on the device.

Unfinished Work

The Android port is at about 90%. The remaining work is polish and edge cases, tracked in issue #20: the Settings screen toggles are not saved and the watch connection status is still a placeholder, some drill descriptions do not match the actual shots per spot, and the phone and watch do not yet recover shots sent while they are disconnected. We also found a bug in the existing Apple Watch app (issue #19): when the iPhone sends the same shot twice in a row, the watch counts only one. Both issues have been moved to Sprint 5.

Completed Issues/User Stories

Here are links to the issues that we completed in this sprint:

- [#15 Implement streak tracking gamification \(Android\)](#)
- [#16 Implement goal progress gamification \(Android\)](#)
- [#17 Port iOS app to Android \(~90%\)](#)
- [#18 Integrate Wear OS companion app with Android phone app](#)
- [#22 Implement XP and level gamification \(iOS\)](#)

Incomplete Issues/User Stories

Here are links to issues we worked on but did not complete in this sprint:

[#19 Apple Watch misses repeated identical shots sent from iPhone](#) — We found this bug while reviewing the watch code during the port; fixing it was not part of the Sprint 4 goals, so it is scheduled for Sprint 5.

[#20 Android/Wear OS parity polish and edge cases](#) — The Sprint 4 target was about 90% of the port; the remaining polish and disconnect handling are planned for Sprint 5.

Team Contributions

Team member	Sprint 4 work	Issues	Story points
Kellam Adams	Android phone app port, Wear OS companion app, phone/watch messaging	#17, #18	14
Melvin Sanare	Streak tracking, goal progress, watch gamification sync, communication logging, emulator testing, GitHub issues and milestones	#15, #16, #18	10
Kaleb Kebede	iOS XP and level system, goal progress	#16, #22	7

Android Feature-Parity Checklist

iOS/watchOS feature	Android/Wear OS status
Dashboard	Done
Drills list and drill details (8 drills)	Done
Workout screen: start/stop, live counts, Quick Log	Done
Court positions for each drill	Done
History	Done
Analytics	Done
Goals	Done (expanded with rings and editing)
Settings	Partial: toggles not saved, watch status is a placeholder
Saving workout data on the phone	Done
Watch splash screen	Done
Watch court view with position marker	Done
Watch swipe shot logging (make, miss, swish)	Done
Watch tap-to-set position in general workouts	Done
Watch workout summary by position	Done
Phone/watch sync: start, stop, shots, drill, position	Done
Recovery after phone/watch disconnect	Not yet (Sprint 5)

Testing and Validation

Test suite	What it checks	Result
<code>StreakCalculatorTest</code>	Streak rules using known workout dates (consecutive days, rest day, two missed days, same-day sessions, best streak)	17/17 pass
<code>GoalTrackerTest</code>	Expected vs. calculated goal progress with known data, week boundaries, FG% minimum shots, once-per-period celebrations	10/10 pass
<code>GamificationPayloadsTest</code>	Phone-to-watch stats and celebration messages	5/5 pass
<code>ModelsTest</code>	Shot codes match the Apple Watch protocol, drill positions, percentages	3/3 pass
<code>ExampleUnitTest</code>	Default Android test	1/1 pass
Total		36/36 pass

Emulator testing (Pixel phone paired with a Wear OS watch):

Ran full workouts with shots logged on the watch and confirmed the counts on the phone.

Changed the phone's date to confirm the streak grows the next day, survives a rest day, and resets after two missed days while the best streak is kept.

Edited goals, completed them, and confirmed the popup appears once per day.

Confirmed every phone/watch message in Logcat under the `SquatchComm` tag.

Measured CPU usage: the idle app dropped from about 50% to 0% after fixing the flame animation.

Code Files for Review

Please review the following code files, which were actively developed during this sprint, for quality:

[Streaks.kt](#)
[Goals.kt](#)
[StreakUi.kt](#)
[GoalsUi.kt](#)
[GamificationUi.kt \(watch\)](#)
[WorkoutApp.kt \(watch\)](#)
[WorkoutScreen.kt \(phone\)](#)
[AppDataStore.swift \(iOS XP\)](#)
[DashboardHomeView.swift \(iOS Player Progress\)](#)

Client Meetings

[September 2, 2026 – semester kickoff with Squatch Sports](#)
[September 17, 2026 – client consultation on Sprint 4–6 deliverables](#)

Responsible Use of Generative AI

How we used it: As the client requested, we used AI tools heavily this sprint. Kellam used AI to convert the iOS and watchOS app into the Android and Wear OS project. Melvin used Claude Code to write the streak, goal, and watch-sync code and their unit tests, to diagnose emulator problems from device logs, and to help organize GitHub issues and documentation.

What it improved: The port and the gamification features were built much faster than the team could have done while learning Android for the first time, and the AI-written unit tests gave us 36 automated checks using known data.

Limitations we found: The AI-generated flame animation built and passed every test but kept the screen redrawing constantly, which made the emulators unresponsive. We only caught it through hands-on testing. AI output also depends on the rules we give it; for example, the team had to decide the streak rules (rest days, same-day sessions) ourselves.

How we verified it: Every AI-assisted feature was reviewed by the team and tested in the phone and watch emulators, and the business rules were checked with unit tests built from known dates and data.

Accountability: All final decisions and deliverables remain the team's responsibility.

Retrospective Summary

Here's what went well:

The Android and Wear OS port came together quickly, and all major flows run in the emulators. Each team member delivered a working gamification or port feature.

Testing with known dates and emulator date changes gave us confidence in the streak and goal logic.

GitHub issues, milestones, and story points now show each member's contribution.

Here's what we'd like to improve:

Coordinate earlier on which platform each feature is built for, so gamification is consistent across Android and iOS.

Test on devices earlier, instead of relying only on builds and unit tests.

Commit and review work in smaller pieces throughout the sprint rather than near the deadline.

Here are changes we plan to implement in the next sprint:

Finish the remaining Android and Wear OS polish and disconnect handling (#20).

Fix the Apple Watch repeated-shot bug (#19).

Improve statistics and chart presentation, and add practice-time tracking.

Integrate and refine the gamification features across platforms based on client feedback.